PROYECTO CHATBOT

LuxDrive ChatBot

Chatbot inteligente para la gestión de reservas

Lenguaje natural • API • Base de datos

Presentación del sistema · Arquitectura y funcionamiento

Juan Camilo Restrepo-Juan José Valbuena-Sebastian Vélez

01 · INTERFAZ

Gradio

02 · INTERPRETACIÓN

Chatbot + Groq

03 · SERVICIOS

FastAPI

04 · PERSISTENCIA

SQLite

¿Qué es LuxDrive?

Una aplicación que permite gestionar reservas mediante mensajes en lenguaje natural.

- El usuario escribe lo que necesita, sin aprender comandos.
- El chatbot interpreta la intención y recopila los datos.
- La API ejecuta las operaciones y consulta la información almacenada.

OBJETIVO

Simplificar la creación, consulta, modificación y cancelación de reservas.

ENFOQUE

Combinar una interfaz conversacional con una API y una base de datos.

IDEA CLAVE

La IA interpreta; el backend valida y ejecuta.

Problema y propuesta de solución

PROBLEMA

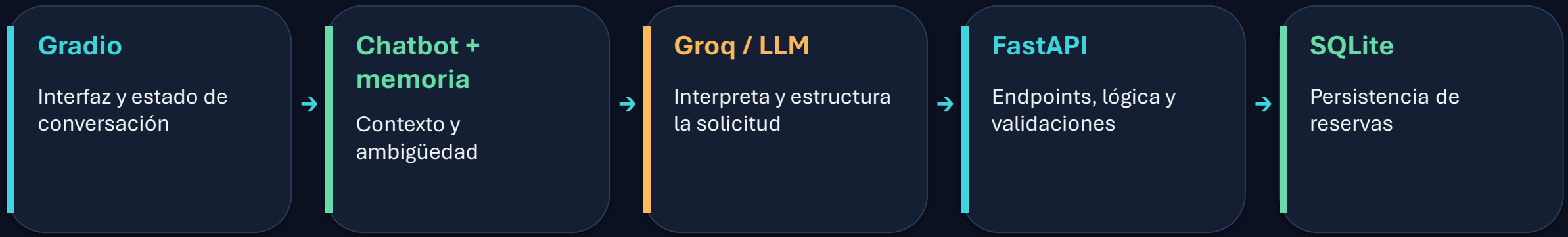
La gestión tradicional de reservas puede requerir formularios, búsquedas manuales y varios pasos para completar una solicitud.

PROPUESTA

Un chatbot que recibe solicitudes en lenguaje natural y las conecta con operaciones reales de gestión de reservas , evitando asi los problemas mencionados anteriormente.

Resultado esperado: una interacción más sencilla, con validaciones centralizadas en el backend.

Arquitectura general



Flujo: usuario → interfaz → interpretación → API → base de datos → respuesta

La separación por módulos facilita el mantenimiento y delimita responsabilidades.

Interfaz y conversación

app_gradio.py

- Recibe mensajes
- Presenta respuestas
- Gestiona confirmación y descarte
- Actualiza el estado

chatbot_memoria.py

- Conserva el historial
- Busca servicios
- Maneja solicitudes ambiguas
- Pide aclaraciones

consulta_llm.py

- Consulta Groq
- Lee intención y plan
- Llama a la API
- Formatea la respuesta

La memoria mantiene el contexto; no reemplaza las validaciones ni la persistencia.

Inteligencia artificial: alcance y límites

LO QUE SÍ HACE

- Interpreta mensajes en lenguaje natural
- Identifica una intención probable
- Organiza datos para una operación
- Ayuda a mantener el contexto

LO QUE NO DEBE HACER

- Modificar directamente la base de datos
- Sustituir la validación del backend
- Decidir disponibilidad sin consultar
- Autorizar acciones sin verificar permisos

La IA propone la operación; el backend controla su ejecución.

API y operaciones CRUD

CREATE	POST /api/reservas	Crear una reserva
READ	GET /api/reservas	Listar reservas
READ	GET /api/reservas/{codigo}	Consultar una reserva
UPDATE	PUT /api/reservas/{codigo}	Modificar una reserva
DELETE*	DELETE /api/reservas/{codigo}	Cancelar una reserva

*La cancelación cambia el estado de la reserva; conserva el historial.

Base de datos y modelo de reserva

SQLite · reservas.db

Almacenamiento local y persistente.

almacen_sqlite.py encapsula consultas SQL y operaciones de lectura y escritura.

DATOS PRINCIPALES DE UNA RESERVA

Código	Identificador
Cliente	Persona asociada
Servicio	Tipo de servicio
Fecha y hora	Inicio de la reserva
Duración	Tiempo estimado
Estado	Activa o cancelada

negocio.json aporta el catálogo de servicios, precios, duraciones y horarios.

Disponibilidad y concurrencia

1 · CONSULTAR

Se buscan reservas activas cuyo intervalo se cruce con el solicitado.

2 · BLOQUEAR

BEGIN IMMEDIATE obtiene el bloqueo de escritura de SQLite al iniciar la transacción.

3 · CONFIRMAR / REVERTIR

commit() confirma; rollback() revierte la transacción ante un error.

Sin una secuencia transaccional, dos solicitudes podrían comprobar el mismo horario libre antes de que alguna lo guarde.

La disponibilidad debe comprobarse dentro de la transacción que protege la escritura.

Recorrido de una solicitud

1	**Usuario escribe**	“Quiero reservar el servicio X mañana a las 10”.
2	**Chatbot interpreta**	Identifica la intención y recopila datos.
3	**Confirmación y API**	La operación estructurada se envía a FastAPI.
4	**Validación y SQLite**	Se comprueba disponibilidad y se guarda si corresponde.
5	**Respuesta**	El resultado vuelve a Gradio.

Si faltan datos o hay ambigüedad, el flujo solicita una aclaración.

Validaciones, errores y límites

DATOS

Verificar que la solicitud incluya la información necesaria y valores válidos.

DISPONIBILIDAD

Consultar las reservas reales antes de guardar cambios.

PERMISOS

Evitar cambios en reservas ajenas requiere autenticación y autorización en el backend.

Confirmar una operación no es lo mismo que autenticar al cliente.

Tecnologías utilizadas

Python

Lenguaje de implementación

Gradio

Interfaz conversacional

Groq / LLM

Interpretación de lenguaje natural

FastAPI

API y endpoints HTTP

SQLite

Persistencia relacional local

La arquitectura separa interfaz, interpretación, lógica de negocio y almacenamiento.

Demostración sugerida

Crear	Solicitar una reserva y confirmar los datos.
Consultar	Buscar una reserva mediante su código.
Modificar	Cambiar fecha u hora y validar disponibilidad.
Cancelar	Cancelar y comprobar el estado.
Ambigüedad	Enviar una solicitud incompleta y observar la aclaración.

Entorno local: interfaz Gradio · API FastAPI · documentación interactiva /docs

Conclusiones

LuxDrive conecta el lenguaje natural con operaciones reales de gestión de reservas.

- La arquitectura modular distribuye las responsabilidades.
- La API centraliza la lógica y las validaciones.
- SQLite conserva las reservas y permite consultarlas.
- La IA mejora la interacción, pero no sustituye los controles del sistema.

Gracias por su atención.